Experiment No-10

```
# Naive Bayes Classifier for Email Spam Detection

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.metrics import classification_report

# Step 1: Create a sample email dataset
emails = [
    "Win a free iPhone now",
    "Congratulations you won a lottery",
    "Claim your free prize today",
    "Get free money immediately",
    "You have won a cash prize",
    "Limited offer buy now",
    "Free entry in a competition",
    "Win money with this offer",
    "Meeting scheduled for tomorrow",
    "Please submit your assignment",
    "Let's have lunch today",
    "Project meeting at 10 AM",
    "Please call me when you are free",
    "Your class notes are ready",
    "Submit the report by Monday",
    "Can we meet tomorrow",
    "The lecture starts at nine",
    "Please check your email"
]

labels = [
```

```
    "Spam", "Spam", "Spam", "Spam", "Spam", "Spam", "Spam", "Spam",
    "Ham", "Ham", "Ham", "Ham", "Ham", "Ham", "Ham", "Ham", "Ham", "Ham"
]

# Step 2: Convert email text into numerical features
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(emails)
y = labels

# Step 3: Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

# Step 4: Create and train the Naive Bayes model
model = MultinomialNB()
model.fit(X_train, y_train)

# Step 5: Predict email categories
y_pred = model.predict(X_test)

# Step 6: Evaluate the model
print("Naive Bayes Email Classification")
print("-------------------------------")
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred, labels=["Ham", "Spam"]))

print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred, zero_division=0))

# Step 7: Test a new email
new_email = ["Congratulations! You won free money"]
new_email_vector = vectorizer.transform(new_email)

prediction = model.predict(new_email_vector)

print("\nNew Email:", new_email[0])
print("Prediction:", prediction[0])
```

Output of the code-

```
Naive Bayes Email Classification
--------------------------------
Accuracy: 66.67 %

Confusion Matrix:
[[2 1]
 [1 2]]

Classification Report:
              precision    recall  f1-score   support

         Ham       0.67      0.67      0.67         3
        Spam       0.67      0.67      0.67         3

    accuracy                           0.67         6
   macro avg       0.67      0.67      0.67         6
weighted avg       0.67      0.67      0.67         6


New Email: Congratulations! You won free money
Prediction: Spam
```